AI研发新范式：基于技术方案全链路生成代码

原创 腾讯程序员 腾讯技术工程 2025年12月12日 01:36 广东 659人



作者：jacksondeng、gcchaoguo 、haoxili 、kxingzhang 、darychen

导语

过往基于tab模式进行代码补全对编码效率提升的天花板较低，使用agent模式生成的代码在功能完整性，可用性，安全以及可维护性又有很多问题导致实际代码采纳率不理想；基于以上矛盾点腾讯广告审核团队迫切需要探索出适合团队并且能面向未来的AI开发范式，帮助团队从需求确认到功能上线全链路提效；本文重点介绍通过“AI技术规范(索引/rules) + 模板化(技术方案/prompt) + MCP工具集成 + AI自我总结”，实现生产力提升的人机协同新范式。

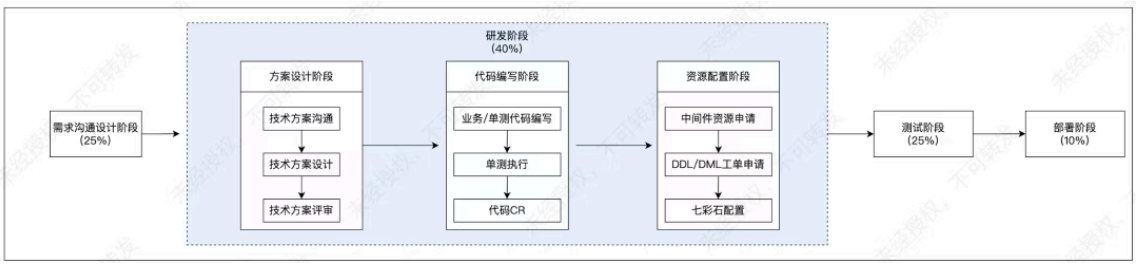
1.背景与挑战

1.1 背景

AI Code正引发一场深刻的软件开发范式变革，其角色已从辅助性的“效率工具”，演进为重塑研发、测试、运维全流程的核心引擎。面对AWS基于规范驱动模型生成代码的Spec范式，Lovable基于自然语言直接生成代码的AI Native范式的推出，以及Cursor、Trae、CodeBuddy等工具的涌现，我们认为AI技术体系已进入可工程化的新阶段。因此，我们亟需探索并确立一套适合广告审核团队特性的AI Code最佳实践范式，以实现研发效率的全面提升。

1.2 现状

审核系统的业务复杂度较高，其研发模式主要基于既有的架构、系统和规范进行迭代演进，且对可用性、可维护性与安全性有着极高要求。每一个需求从设计到上线的全流程，通常都涉及跨越多角色、多平台和多模块的庞杂交付工作。目前，研发阶段占全流程的40%，此阶段以人工设计开发为主导，可以利用AI提效的环节和空间较大。



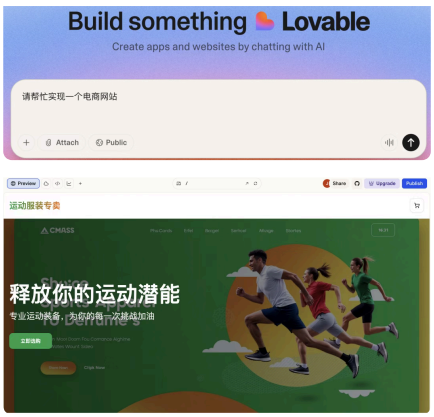
1.3 挑战

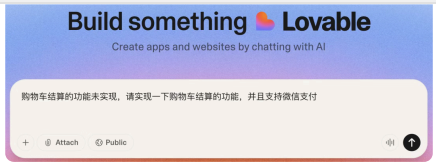
近年来，AI编程工具虽已在代码生成领域取得长足进步，展现出了巨大潜力。然而，在业务逻辑复杂、技术架构复杂的企业级工程“深水区”，现有AI编程工具仍需应对来自业务、技术架构与模型自身的三重挑战：

具体挑战	挑战描述
业务挑战	大模型在应对**跨模块协同、高复杂度业务逻辑及存量系统迭代**时，因缺少对整体业务知识及背景的输入，生成的代码易存在割裂、冗余、适配不足问题，具体如下： • 需求跨多模块时，大模型缺少各模块的依赖和调用关系，生成的代码比较割裂。比如人机协同需求跨多个模块（机审模块、人审模块、模型组模块、数据模块） • 业务复杂度高，大模型缺少背景知识理解困难，生成的代码比较冗余 • 基于已有系统迭代，大模型因缺乏架构认知，生成的代码适配不足，与已有系统难以兼容
技术架构挑战	大模型因缺少**安全规范、代码结构**与**设计方案**，导致架构知识及背景输入匮乏，生成的代码易存在安全漏洞、质量缺陷、架构偏离问题，具体如下： • 系统涉及敏感信息时，大模型缺少安全约束，可能导致信息泄露等安全事故，如账号、密码等 • 大模型缺少仓库目录及代码层级结构等信息，大模型在难以精准定位目标目录、映射其设计语义，从而限制了代码生成的准确性 • 大模型缺少业务逻辑的定义，难以将需求其转化为清晰、可执行的实现路径 • 大模型对闭源或私有组件的内部规范存在认知盲区，难以生成可用的代码
模型挑战	大模型在生成代码时候，存在**token配额限制，调用次数限制**，以codebuddy生态为例，具体如下： • token限制：目前使用的大模型，token的额度是有限的，并且每天的额度是动态调整的 • 工具限制：大模型对于MCP工具的访问次数有限制，一次对话一般限制在几十次左右

2.业界AI Code实践范式

当前，AI编程领域的研发范式层出不穷，呈现快速演进与多元探索的格局。其中，AWS的Spec范式与Lovable的AI Native范式因其鲜明的技术理念与显著的应用效果而备受业界关注。为系统把握其核心思路、优势差异及适用场景，我们针对这两类代表性研发范式展开了专项调研与对比分析。

特性维度	AWS Kiro Spec范式	Lovable AI native范式
核心思路	基于规范驱动模型生成代码	基于自然语言直接生成代码
目标用户	专业开发者、技术团队	创业者、产品经理、无代码背景的创作者
工作流程	需求文档 → 设计文档 → 任务清单 → 代码实现  ``` # Requirements Document ## Introduction 本文档定义了知识库会话功能的详细需求。该功能旨在为广告审核系统提供智能问答能力，支持用户通过企微机器人或ifram 知识库会话功能将集成到现有的广告审核系统中，采用分层架构设计，包含数据访问层、业务逻辑层和控制器层，确保与现有 ## Requirements ### Requirement 1: 知识库会话数据管理 **User Story:** 作为系统管理员，我希望能够完整记录用户与知识库的交互会话数据，以便进行数据分析和服务优化。 #### Acceptance Criteria 1. WHEN 用户通过任何渠道（企微机器人或iframe嵌入）提交问题 THEN 系统 SHALL 创建新的会话记录并保存到数据 2. WHEN 模型生成回复内容 THEN 系统 SHALL 将回复内容与原始问题一起保存到会话记录中 3. WHEN 会话记录被创建或更新 THEN 系统 SHALL 自动设置创建时间和更新时间字段 4. IF 用户提供了原始图片 THEN 系统 SHALL 保存图片URL到会话记录中 5. WHEN 保存会话数据 THEN 系统 SHALL 确保所有必填字段（session_id, question_time, user_name, user ### Requirement 2: 会话查询和检索功能 **User Story:** 作为业务人员，我希望能够查询和检索历史会话记录，以便分析用户行为和优化服务质量。 #### Acceptance Criteria 1. WHEN 请求查询单个会话详情 THEN 系统 SHALL 根据session_id返回完整的会话信息 2. WHEN 请求查询会话列表 THEN 系统 SHALL 支持按用户名、时间范围、渠道类型等条件进行筛选 3. WHEN 执行列表查询 THEN 系统 SHALL 支持分页功能，默认每页返回20条记录 4. WHEN 查询不存在的会话ID THEN 系统 SHALL 返回适当的错误信息 5. WHEN 执行查询操作 THEN 系统 SHALL 在500毫秒内返回结果 ```	自然语言描述 → AI生成并预览应用 → 反馈调整 

特性维度	AWS Kiro Spec范式	Lovable AI native范式
		
核心优势	高可控性，适合企业级应用（补充）	极速上手、门槛极低，适合营销活动和展示demo产品
主要局限	● 需求/设计/任务清单的格式复杂、要求严格、门槛高 ● 研发流程工具集成不够完善，比如类tapd工具、iwiki工具、DDL等	● 不适合已有系统的迭代 ● 不适合定制化的需求实现 ● 不适合复杂度高业务

3.审核AI Code实践范式

3.1AICode范式定义

基于行业经验，围绕“如何让AI更理解业务”，生成更可控准确、更完整简洁的代码，审核在现有系统基础上，探索并沉淀了审核业务场景的AI Code范式，核心是要给模型提供更全更准的业务&技术知识，告诉模型：“按什么规范、在哪里、做什么、怎么做”，并让模型告诉我们“做了啥”。

① 什么规范：指需遵循的技术规范，包括安全规范、设计原则、编码风格、约束条件等，确保生成的代码不仅能work、而且符合技术标准

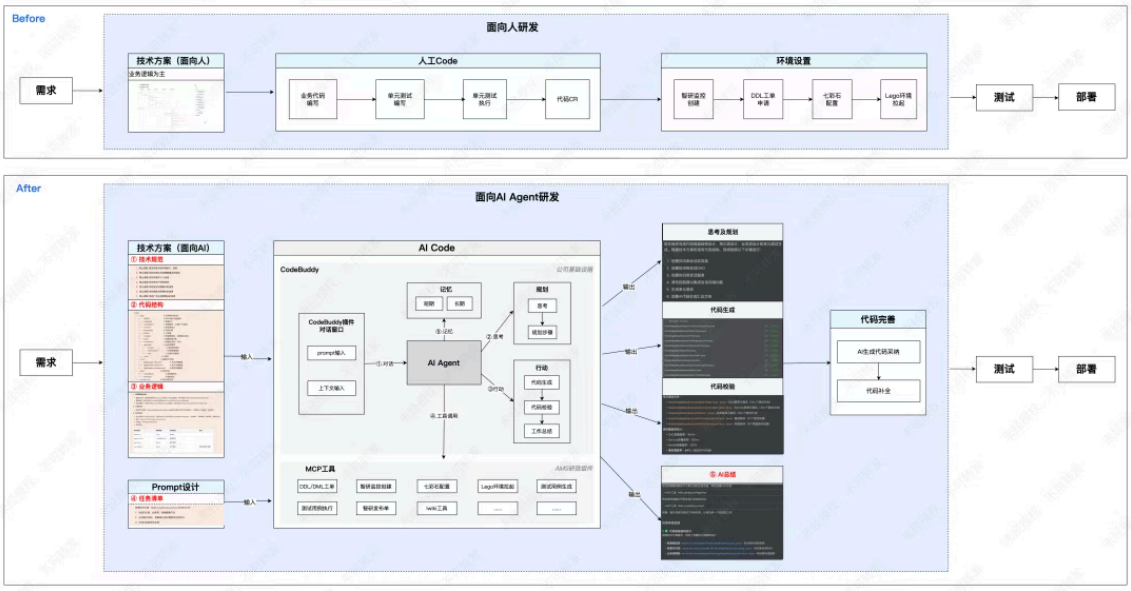
② 在哪里：指现有的代码结构，包括代码仓库目录、代码层级结构、类/方法描述等，辅助模型精准定位代码路径、精确映射设计语义，确保生成的代码位置准确、架构合理

③ 做什么：指需实现的业务逻辑，包括概念定义、功能描述、代码生成范围等，辅助模型精准理解业务需求，精确生成功能代码（细致程度决定生成准确度）

④ 怎么做：指需完成的任务清单及验收标准，指导模型严格按照任务要求逐步实现，并按标准进行检查，确保生成的代码更可控

⑤ 做了啥：指AI总结，让模型在生成代码后，做个自我总结并存档，方便人工review生成的代码是否符合预期，且方便后续维护

结合原有研发流程，我们提炼的AI Code核心范式可概括为：技术方案（①②③） → Prompt（④） → 生成代码 → AI总结（⑤）。该范式以技术方案为起点和依据，通过Prompt提示词作为关键转化，驱动代码生成这一核心环节，并以AI总结完成闭环，整个流程由Agent协同多种MCP工具一站式完成。



我们构建的AI Code开发范式立足于两大支柱：一是稳固的技术基座，这依赖于对AI编程工具、底层大模型及效能度量体系的审慎选型；二是标准化的执行流程，其核心在于将技术规范、方案设计及Prompt指令进行模板化改造。二者共同构成了范式落地的基石。接下来将详细说明我们在技术选型与流程标准化方面的具体实践。

3.2AICode技术选型

- **AI Code配套设施**：IDE + CodeBuddy插件 + 公司先进大模型 + MCP工具
- **AICode度量工具**：eplus平台

3.2.1AI编程工具选型

CodeBuddy在腾讯内生态较为活跃，功能迭代速度快，内部免费、安全性高，而且内部集成了混元、deepseek等模型，可以灵活适配不同场景的需求，所以我们最终选择CodeBuddy作为我们的AI编程工具。

工具	工具简要描述	优点	缺点	结论
Cursor	一款基于VS Code的AI原生IDE，以其强大的多文件理解和生成能力著称，深度集成Claude、GPT等顶级模型	• 多模型自由切换，适应复杂项目需求 • 兼容VS Code插件生态，学习成本低	• 专业版订阅费用高（$20/月） • 国内访问延迟明显，部分功能访问受限	放弃，数据不安全，费用高昂
CodeBuddy	CodeBuddy插件，基于腾讯混元+DeepSeek等模型，提供Agent模式、MCP工具库集成能力	• 多模型可自由切换，适应复杂项目需求 • 生态活跃，功能迭代快，内网版免费	• 本地部署需企业专享版（腾讯员工忽略） • 暂不兼容NetBeans等小众IDE	采纳，安全性高，生态活跃，迭代快
Trae	AI原生IDE，支持Builder模式端到端生成项目、实时预览、跨文件重构，支持deepseek、豆包Pro等模型	• Trae的Builder模式可自动拆解任务→生成代码→运行测试 • 多模型自由切换，灵活适配不同场景需求	• 复杂逻辑生成不稳定，业务逻辑复杂时需人工干预 • 在免费模式下，高频使用时可能需要排队	放弃，数据不安全

3.2.2 大模型的选择

在审核日常的业务需求场景下，使用AI Agent生成代码验证过程中，采用公司先进模型 + iwiki的mcp工具结合，作为我们的代码生成的模型底座，代码行采纳率大于50%。

3.2.3 度量工具的选型

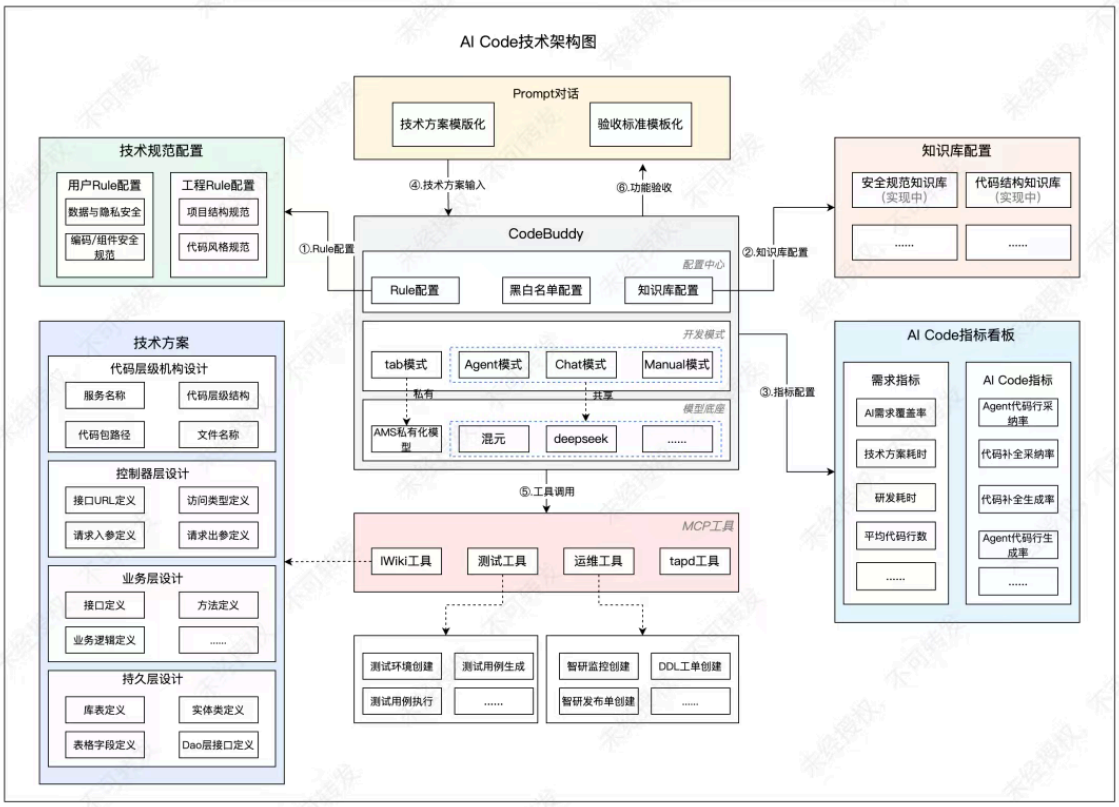
一套优秀的AI代码度量工具，是有效评估AI生成代码成效、设定可量化目标并推动持续优化的基石。它能够为团队提供清晰的观测视角，例如eplus平台目前已支持对**活跃用户数、代码补全采纳率、代码补全生成率、Agent代码行采纳率**等关键指标进行监控，从而帮助团队精准掌握AI编码的应用效果，并指引其向既定目标稳步推进，具体数据指标类型如下图所示：



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
690篇原创内容
公众号

3.3AICode过程标准化

为了让AI生成更可控、更准确、更完整简洁的代码，我们给AI制定了一套标准的技术规范，在技术方案上从面向人转变到面向AI，同时将Prompt提示词标准化，方便给大模型发出清晰的指令，最后让大模型给出自我总结，以便人工检查和后续维护。目前我们已将这套标准的AI研发流程推广到审核中心，AI实现的需求覆盖率达到70%+，同时Agent代码行采纳率在50%以上，研发节约时长比例达到30%+。



3.3.1 技术规范制定

为应对审核系统多语言、多框架的复杂技术栈及差异化的安全要求，需为AI代码生成制定涵盖安全、工程与代码的规范性标准，以确保输出代码除保证可用性之外，还符合项目的长期技术约束与质量目标，部分规范示例如下：

规范维度	核心要点	简要说明
安全规范	通用安全编码	对所有输入输出进行校验和转义，防范注入攻击，使用预编译SQL
	数据与隐私安全	• 禁止在日志、配置中记录敏感信息，对敏感数据脱敏或加密 • 禁止读取/修改项目中的所有账号、密码 • 禁止读取/修改项目中风控策略相关的信息 • 禁止读取/修改项目中个人信息 • 禁止读取/修改知识产权的信息 • 禁止读取/修改安全凭据相关的信息 • 禁止读取/修改商业机密相关的信息 • 禁止读取/修改广告主资质相关的信息
	依赖组件安全	禁止使用已知漏洞的组件，如fasterjson版本需>= 1.2.83 、log4j版本需>= 2.17.0等
工程规范	项目结构规范	项目中严格按照Controller-> Service-> Repository -> Model 分层架构来设计
	依赖管理	明确定义依赖项版本，避免冲突，统一管理公司内部私有库依赖
	代码质量管理	提交前自动触发代码风格、安全漏洞、简单Bug的静态检查
	部署与运行	配置与代码分离，避免硬编码；添加/修改接口需要有日志记录和监控告警
代码规范	代码风格与格式	为Java、Go、Python、Scala等多种语言分别制定编码约定，遵循各语言主流风格指南，如Java的Google Style，Python的PEP8等
	文档与注释	公共API、复杂核心算法必须添加注释。注释需说明“为何如此做”

3.3.2 技术方案模版化

为适配AI编程，技术方案的设计思路需进行根本性的范式转型，即从易于人类理解转变为利于AI识别与执行。为实现这一目标，我们需要推动技术方案的标准化与模块化，将其固化为可复用的模板。开发人员仅需根据具体需求对模板进行少量调整，即可快速生成高质量方案，这将

极大降低研发人员采用AI编程的门槛，并为我们在全中心推广奠定了基础，目前我们将技术方案分为：代码层级结构设计、控制器层设计、业务逻辑层设计、持久层设计，具体每一层的设计如下所示：

- 代码层级结构模版化设计（structrue）：代码层级结构设计的核心目标之一，是建立一套明确的代码组织契约。它应清晰定义在哪个层级、哪个模块下，创建何种类型和职责的文件。这为AI生成代码提供了规范性指引，更为重要的是，它为开发团队后续的修改、阅读和维护提供了精准的定位地图，从而有效降低维护成本。

代码层级结构模版截图	示例截图
1.1 代码层级结构设计 1.1.2 A服务 • constant（常量定义）：在A服务中的constant中添加xxx的java类。 • controllers（控制器层）：在A服务的controllers中添加xxx的java类。 • enums（枚举类定义）：在A服务的enums中添加xxx的java类。 • exception（异常处理）：在A服务的exception中添加xxx的java类。 • repository（数据访问层）：在A服务的repository中添加xxx包路径，同时添加xxxDao的java类。 • services（业务逻辑层）：在A服务的services中添加xxx包路径，同时添加xxxService的java类。 • …… 1.1.3 B服务 • constant（常量定义）：在B服务中的constant中添加xxx的java类。 • controllers（控制器层）：在B服务的controllers中添加xxx的java类。 • enums（枚举类定义）：在B服务的enums中添加xxx的java类。 • exception（异常处理）：在B服务的exception中添加xxx的java类。 • repository（数据访问层）：在B服务的repository中添加xxx包路径，同时添加xxxDao的java类。 • services（业务逻辑层）：在B服务的services中添加xxx包路径，同时添加xxxService的java类。 • …… （ps：新增java类需要在代码层级结构说清楚在哪新增，修改不需要）	1、代码层级结构设计 • 控制器层：在maat服务的的controller中添加knowlege包路径，同时添加KnowledgeBaseSessionControllerV1的java类。 • 业务逻辑层：在maat服务的的services中添加knowlege包路径，同时添加KnowledgeBaseSessionServiceV1的java类。 • 数据访问层：在maat服务的repository中添加knowlege包路径，同时添加KnowledgeBaseSessionDaoV1的java类。

- 控制器层模版化设计（controller）：控制器层主要是为前端层设计API，如果需要对前端提供API，需向大模型明确其技术契约，包括协议、路径、参数、响应格式等核心约束条件，以确保生成的接口代码可直接投入使用。

控制器层模版截图	示例截图
1.2 控制器层设计 1.2.1 A服务 • xxx控制器：在xxx控制器中，实现xxxx内容的插入、详情查询、列表查询、更新方法。 （ps：如果想要代码生成的更加精准，可以描述到方法级别） 1.2.2 B服务 • xxx控制器：在xxx控制器中，实现xxxx内容的插入、详情查询、列表查询、更新方法。 （ps：如果想要代码生成的更加精准，可以描述到方法级别）	2、控制器层设计 • 会话管理控制器： ◦ 详情查询接口：在maat服务的KnowledgeBaseSessionControllerV1类中实现知识库会话内容的详情查询接口 ▪ 接口URL：/api/v1/knowlegebase/getSessionById ▪ 请求类型：Post ▪ 请求入参：sessionId ▪ 请求出参：KnowlegeBaseSessionV1类中所有字段 ◦ 列表查询接口：在maat服务的KnowledgeBaseSessionController类中实现知识库会话内容的列表查询接口 ▪ 接口URL：/api/v1/knowlegebase/getSessionList ▪ 请求类型： Post ▪ 请求入参：userName、sessionId、createTime、pageNum、pageSize ▪ 请求出参：KnowlegeBaseSessionV1类中所有字段

- 业务逻辑层模板化设计（service）：service层主要写一些核心的业务逻辑代码，修改Service层逻辑时，需向大模型精准指定目标类、方法及具体的功能变更点。

业务逻辑层模版截图	示例截图
1.3 业务层设计 1.3.1 A服务 • xxx服务：在xxx类中实现xxx内容的插入、详情查询、列表查询、更新接口。 （ps：如果想要代码生成的更加精准，可以描述到方法级别） 1.3.2 B服务 • xxx服务：在xxx类中实现xxx内容的插入、详情查询、列表查询、更新接口。 （ps：如果想要代码生成的更加精准，可以描述到方法级别）	3、业务层设计 • 知识库会话服务：在maat服务的KnowledgeBaseSessionServiceV1类中实现知识库会话内容的插入、详情查询、列表查询、更新接口。

● 持久层模版化设计（repository）：持久层主要是写一些存储相关的接口，修改或者新增持久层代码的时候，需向大模型提供具体的Dao接口、数据实体类、库表名称、表格字段等信息

持久层模版截图	示例截图
	

3.3.3 prompt模版化

为提升大模型对指令的解析精度，同时降低研发人员的使用门槛与操作成本，我们通过构建一套规范化、可复用的Prompt模板，实现了提示词的标准化管理。

```
参考技术方案：{iwiki链接}
1、生成持久层、业务层、控制器层代码
2、生成单元测试，保障单元测试覆盖率达到80%
3、在{iwiki链接}的文档目录下，创建一个子文档为【AI自我总结】。
```

3.3.4 AI自我总结

在AI生成代码后，强制其进行自我总结并归档。该文档直接服务于人工代码审查（快速验证是否符合预期）与长期项目维护（留存关键决策上下文），是提升AI代码可用性的关键步骤。



4.实践总结

4.1进展及效果

审核中心已全面推广该范式，人员覆盖率100%、需求覆盖率70%+、Agent代码行采纳率在50%以上。



4.2 困难/不适用点

虽然该范式已全面推广，但大模型本身依然存在发散和遗漏的问题，同时在项目实现上也存在大模型不适合及知识盲区的问题。

问题分类	描述举例	当前解决方案
大模型幻觉 （思维发散、记忆遗漏）	思维发散：大模型有时会“自作聪明”和“画蛇添足”， 修改或生成一些无关紧要的代码 如人审精标需求只需要插入功能，大模型会生成完整的CRUD	技术方案中限定实现范围
	记忆遗漏：prompt中描述任务顺序会影响大模型生成代码的完整性 如先生成业务层代码、再生成存储层代码时，可能会遗漏，因为业务层依赖存储层接口	严格按照逻辑依赖顺序描述任务
非功能性需求 （当前不适合）	数据问题：如分库分表逻辑实现、数据清理等需求，暂不适合AI实现	人工实现
	安全问题：如代码仓库中修改账号和密码的诉求，暂不适合AI实现	人工实现
	紧急问题：如遇到线上紧急问题，需快速修复和止损，暂不适合AI实现	人工实现
历史或闭源项目 （知识盲区）	非开源项目：如腾讯内部闭源的sdk接入，大模型没法识别	输入闭源SDK实现模板供参考
	老旧项目：如依赖的一些陈旧的技术栈，大模型不了解	人工实现
Agent/工具限制 （客观条件）	Agent限制：在涉及多模块的复杂大需求上，因受token配额、工具调用次数等限制，难以保障需求实现的完整性和准确性 如人机协同需求，涉及样本管理、预处理等多个模块，因需要更多上下文，导致token和工具调用次数耗尽，进而中断代码生成过程	拆分合适颗粒度，确保输入完整精炼 将单个Project（独立可编译模块）作为单次AI Code范围

5.未来展望

当前AI Code的应用价值已在编码环节得到验证，为将其增效能力从“单点突破”升级为“全局赋能”，我们下一步将着力打通开发、测试、部署环节的全链路协同，并重点探索“需求->技术方案”的端到端一站式研发的可行性。

实践方向	关键事项描述
打通Codebase知识库	沉淀技术方案中技术规范和代码结构等公共知识到大仓，使业务研发仅关注业务逻辑部分，同时让范式更加简洁标准，具备可推广性
打通开发、测试及部署全链路	以技术方案为载体，**打通测试和运维MCP工具**，实现一站式Agent研发 • **测试工具打通**：提交代码后，自动拉起lego环境、自动执行测试用例等，实现研发到测试的Agent协同 • **运维工具打通**：测试完成后，自动发起DDL工单、自动配置七彩石、自动创建监控视图等，实现研发到部署交付的Agent协同
探索“需求->技术方案”可行性	参考AWS-Spec、Lovable-AI Native等业界实践，探索从TAPD需求到技术方案的实现路径，逐步向端到端的一站式研发模式演进

结语

本文阐述的AI研发范式：技术方案模板化→Prompt模板化→代码生成→AI总结，是基于广告审核场景总结提炼出来的实践范式。该范式可能未必普适，不过其核心方法论——将人工分析的需求，通过模板转化为面向AI的技术方案与清晰指令，最终驱动代码生成——具有重要的可迁移价值。我们希望这一抛砖引玉的研发范式能为寻求利用AI生成高质量代码的团队提供有益参考。



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
690篇原创内容

公众号



阅读 2.3万

留言

写留言

还没有留言，来写首评